Inteligencja obliczeniowa w analizie danych | Inżynieria i Analiza Danych

Projekt Algorytm Genetyczny

Mateusz Bugdol

Nr indeksu: 419719

Grupa ćwiczeniowa: 1

1. Cel projektu

Celem zadania jest praktyczne sprawdzenie, jak działają algorytmy ewolucyjne na przykładzie problemu komiwojażera (TSP - Traveling Salesperson Problem). Projekt ma pokazać, że wykorzystując mechanizmy inspirowane naturalną ewolucją – takie jak dobór celowy, krzyżowanie i mutacja – jesteśmy w stanie z dużą dokładnością wyznaczyć wysoce zoptymalizowaną trasę łączącą zadane punkty. Wyznaczenie jej metodą analityczną (przeglądem zupełnym) jest niezwykle trudne ze względu na ogromną liczbę możliwych kombinacji (silnia z liczby miast). Kluczowym aspektem jest zbadanie, jak proces ewolucji w kolejnych pokoleniach wpływa na minimalizację całkowitego dystansu i zbieżność algorytmu do optymalnego rozwiązania.

2. Opis zadania

Zadanie polega na wyznaczeniu najkrótszej trasy łączącej zadaną listę miast. Każde miasto musi zostać odwiedzone dokładnie raz, a trasa musi kończyć się w punkcie startowym.

Kroki wykonywania programu:

- Pobranie rzeczywistych współrzędnych geograficznych (długość i szerokość) wybranych miast z wykorzystaniem zewnętrznego API.
- Inicjalizacja populacji początkowej – wygenerowanie zadanej liczby losowych permutacji tras (osobników).
- Obliczanie funkcji przystosowania (fitness) – wyznaczanie całkowitego dystansu dla każdej trasy w populacji.
- Ewolucja przez zadaną liczbę pokoleń: selekcja (wybór najlepszych osobników do reprodukcji), krzyżowanie (tworzenie potomków łączących cechy rodziców) oraz mutacja (losowa zamiana miast w celu uniknięcia wpadnięcia w minimum lokalne).
- Zapisywanie najlepszych wyników z każdego pokolenia w celu śledzenia postępu zbieżności algorytmu.

3. Matematyczne i logiczne podstawy algorytmu

Aby wyznaczyć najkrótszą trasę, algorytm genetyczny mapuje problem geometryczny na pojęcia z zakresu genetyki:

- Gen (Miasto): Pojedynczy punkt na trasie reprezentowany przez współrzędne przestrzenne.
- Chromosom (Osobnik): Kompletna trasa, czyli wektor będący unikalną permutacją wszystkich badanych miast.
- Funkcja przystosowania (Fitness): Wartość oceniająca jakość osobnika. Dystans między miastami liczymy korzystając z odpowiednich operacji przestrzennych. Wartość fitness to odwrotność całkowitego dystansu trasy: $Fitness = \frac{1}{Dystans}$. Maksymalizacja tej funkcji gwarantuje minimalizację pokonywanej odległości.
- Prawdopodobieństwo krzyżowania i mutacji: W procesie Monte Carlo wykorzystywaliśmy czystą losowość do trafień, natomiast tutaj losowość steruje ewolucją. Para rodziców wymienia ze sobą "geny" tworząc nową trasę dbając o brak powtórzeń miast. Dodatkowo, z pewnym małym prawdopodobieństwem nowa trasa ulega mutacji (losowej zamianie dwóch miast na trasie), co zapobiega utknięciu w rozwiązaniach suboptymalnych.

4. Implementacja i wizualizacja procesu optymalizacji

Przechodzimy do przygotowania danych geolokalizacyjnych oraz budowy silnika algorytmu genetycznego. Całość uzupełnia dynamiczna wizualizacja najlepszych tras z kolejnych generacji, co ostatecznie złoży się na animację obrazującą postęp ewolucji.

4.1. Załadowanie bibliotek

- Requests: Do komunikacji z API OpenStreetMap w celu pobierania dokładnych szerokości i długości geograficznych zdefiniowanych miast.
- Pandas i GeoPandas: Narzędzia do tworzenia, przetwarzania i wektoryzacji przestrzennych struktur danych.
- Shapely.geometry: Umożliwia budowanie i operowanie na obiektach geometrycznych, takich jak punkty miast i łączące je ciągi linii.
- Matplotlib.pyplot i PIL (Image): Główny silnik do rysowania map (miast i ścieżek) oraz składania wygenerowanych wykresów w gotową animację w formacie .gif.
- NumPy, Random, Itertools: Zapewniają szybką obsługę matematyczną oraz wprowadzają niezbędną w algorytmach ewolucyjnych losowość (szansa na krzyżowanie, mutację czy dobór rodziców).

In [1]:
```
import requests
import time
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import random
import geopandas as gpd
from shapely.geometry import Point, MultiLineString, shape
import os
from PIL import Image
from matplotlib.lines import Line2D
import itertools
```

4.2. Szczegółowy opis klasy TSPGenetic

Klasa `TSPGenetic` stanowi główny silnik algorytmu ewolucyjnego rozwiązującego problem komiwojażera (TSP). Hermetyzuje ona zarówno etap pobierania danych geoprzestrzennych, jak i całą logikę ewolucji. Poniżej znajduje się opis poszczególnych elementów i metod klasy:

Inicjalizacja i dane:

- `__init__(self, cities_list)` : Konstruktor klasy. Przyjmuje listę miast docelowych. Inicjalizuje puste struktury danych na populację, historię i najlepsze trasy. Automatycznie wywołuje metodę budującą siatkę połączeń między miastami (`_build_gdf`).

Pobieranie i przetwarzanie danych przestrzennych:

- `_get_coordinates(self, city)` : Komunikuje się z darmowym API Nominatim (OpenStreetMap), aby dla podanej nazwy miasta zwrócić jego współrzędne geograficzne (szerokość i długość).
- `_get_road_data(self, lat1, lon1, lat2, lon2)` : Odpytuje API OSRM (Open Source Routing Machine) o rzeczywistą odległość drogową oraz geometrię (kształt trasy) między dwoma zadanymi punktami.
- `_build_gdf(self)` : Buduje macierz odległości. Tworzy obiekt `GeoDataFrame` zawierający wszystkie miasta, a następnie iteruje po każdej parze miast, wywołując `_get_road_data` . Wynikiem jest pełna wiedza o dystansach i ścieżkach pomiędzy wszystkimi punktami.
- `_get_route_geometry(self, route_indices)` : Na podstawie podanej kolejności odwiedzania miast (indeksów), łączy pojedyncze odcinki dróg w jedną, ciągłą ścieżkę przestrzenną, co jest niezbędne do jej późniejszego narysowania na mapie.

Logika Algorytmu Genetycznego:

- `_calculate_fitness(self, route)` : Oblicza funkcję przystosowania dla danego osobnika (trasy). Sumuje odległości między kolejnymi miastami, a następnie zwraca jej odwrotność ($1/dystans$). Im krótsza trasa, tym większy wynik fitness.

- `_create_initial_population(self)` : Tworzy pierwsze pokolenie (populację zerową) poprzez wygenerowanie zadanej liczby całkowicie losowych permutacji tras.
- `_selection(self, fitness_scores, method, tournament_size)` : Odpowiada za dobór naturalny. Wybiera osobników do reprodukcji z wykorzystaniem jednej z wybranych metod:
  - *tournament (turniejowa)*: Losuje kilku kandydatów i wybiera z nich tego z najlepszym wynikiem.
  - *roulette (ruletka)*: Szansa na wybór jest proporcjonalna do wartości fitness osobnika.
  - *ranking*: Szansa na wybór opiera się na pozycji w rankingu wszystkich tras, a nie na ich absolutnej wartości fitness.
  - *simple*: Całkowicie losowy wybór rodzica.
- `_crossover(self, parent1, parent2)` : Operator krzyżowania (współdzielenia genów). Losuje fragment trasy od pierwszego rodzica, a pozostałe brakujące miasta uzupełnia w kolejności ich występowania u drugiego rodzica (tzw. Order Crossover), co gwarantuje brak powtórzeń miast na trasie dziecka.
- `_mutate(self, route)` : Operator mutacji. Z określonym, niewielkim prawdopodobieństwem zamienia miejscami dwa losowe miasta na trasie potomka. Pozwala to na eksplorację nowych rozwiązań i zapobiega przedwczesnemu utknięciu algorytmu w lokalnym minimum.

Wykonywanie ewolucji i ekstrakcja wyników:

- `run(self, ...)` : Główna pętla algorytmu genetycznego. Inicjalizuje populację, a następnie przez zadaną liczbę pokoleń (`generations`) ocenia osobniki, zapisuje najlepsze wyniki (co 50 pokoleń zapisuje też pełną geometrię do wizualizacji) i tworzy nowe pokolenie przy pomocy selekcji, krzyżowania oraz mutacji (z zachowaniem zasady elityzmu - najlepszy osobnik zawsze przechodzi dalej).
- `get_history_gdf(self)` : Zwraca obiekt `GeoDataFrame` zawierający historyczne etapy optymalizacji trasy. Te dane służą następnie do wygenerowania klatek animacji.
- `get_results(self)` : Zwraca obiekt `GeoDataFrame` z ostatecznym, absolutnie najlepszym znalezionym wynikiem z całego procesu ewolucji.

In [ ]:
```
class TSPGenetic:
    SELECTION_METHODS = ["tournament", "simple", "roulette", "ranking"]

    def __init__(self, cities_list):
        self.city_list = cities_list
        self.road_geometries = {}
        self.citiesGDF = self._build_gdf()
        self.size = len(self.citiesGDF)
        self.best_route = None
        self.history = []
        self.population = []
        self.best_geometries_history = []

    def _get_coordinates(self, city):
        url = f"https://nominatim.openstreetmap.org/search?q={city}&format=json"
        headers = {"User-Agent": "TSP_Genetic_App/1.0"}
```

```python
        try:
            response = requests.get(url, headers=headers)
            data = response.json()
            if data:
                return float(data[0]["lat"]), float(data[0]["lon"])
        except Exception:
            pass
        return None, None

    def _get_road_data(self, lat1, lon1, lat2, lon2):
        url = (
            f"http://router.project-osrm.org/route/v1/driving/"
            f"{lon1},{lat1};{lon2},{lat2}?overview=full&geometries=geojson"
        )
        try:
            r = requests.get(url)
            data = r.json()
            if data["code"] == "Ok":
                distance = data["routes"][0]["distance"] / 1000.0
                geometry = data["routes"][0]["geometry"]
                return distance, geometry
        except Exception:
            pass
        return 999_999.0, None

    def _build_gdf(self):
        data = []
        for city in self.city_list:
            lat, lon = self._get_coordinates(city)
            if lat is not None:
                data.append({
                    "city": city,
                    "latitude": lat,
                    "longitude": lon,
                    "geometry": Point(lon, lat),
                })
            time.sleep(1.1)

        gdf = gpd.GeoDataFrame(data, crs="EPSG:4326")
        for city_name in self.city_list:
            gdf[city_name] = 0.0

        for i, row_start in gdf.iterrows():
            for j, row_end in gdf.iterrows():
                if i == j:
                    continue
                dist, geom = self._get_road_data(
                    row_start["latitude"], row_start["longitude"],
                    row_end["latitude"], row_end["longitude"],
                )
                gdf.at[i, row_end["city"]] = dist
                self.road_geometries[(i, j)] = geom
                time.sleep(0.1)
        return gdf

    def _get_route_geometry(self, route_indices):
        full_path = []
        for k in range(len(route_indices)):
            i = route_indices[k]
            j = route_indices[(k + 1) % len(route_indices)]
```

```
            geom = self.road_geometries.get((i, j))
            if geom:
                full_path.append(shape(geom))
        return full_path

    def _calculate_fitness(self, route):
        total_distance = 0.0
        for i in range(self.size):
            u_idx = route[i]
            v_city_name = self.citiesGDF.iloc[route[(i + 1) % self.size]]['city'
            total_distance += self.citiesGDF.loc[u_idx, v_city_name]
        return 1.0 / total_distance if total_distance > 0 else 0

    def _create_initial_population(self):
        population = []
        base = list(range(self.size))
        for _ in range(self.population_size):
            route = base[:]
            random.shuffle(route)
            population.append(route)
        return population

    def _selection(self, fitness_scores, method="tournament", tournament_size=3)
        if method == "simple":
            return random.choice(self.population)[:]
        elif method == "tournament":
            indices = random.sample(range(len(self.population)), tournament_size
            best_idx = indices[np.argmax([fitness_scores[i] for i in indices])]
            return self.population[best_idx][:]
        elif method == "roulette":
            total = sum(fitness_scores)
            if total == 0:
                return random.choice(self.population)[:]
            probs = [f / total for f in fitness_scores]
            idx = np.random.choice(len(self.population), p=probs)
            return self.population[idx][:]
        elif method == "ranking":
            ranked = np.argsort(fitness_scores)
            n = len(ranked)
            weights = np.arange(1, n + 1, dtype=float)
            probs = weights / weights.sum()
            idx = np.random.choice(ranked, p=probs)
            return self.population[idx][:]
        return random.choice(self.population)[:]

    def _crossover(self, parent1, parent2):
        selected = np.random.choice(self.size, size=2, replace=False)
        start, end = sorted(selected)
        child = [None] * self.size
        child[start:end] = parent1[start:end]
        existing_cities = set(child[start:end])
        remaining_cities = [city for city in parent2 if city not in existing_cit
        r_idx = 0
        for i in range(self.size):
            if child[i] is None:
                child[i] = remaining_cities[r_idx]
                r_idx += 1
        return child

    def _mutate(self, route):
```

```
        if np.random.rand() < self.mutation_rate:
            idx1, idx2 = np.random.choice(self.size, size=2, replace=False)
            route[idx1], route[idx2] = route[idx2], route[idx1]
        return route

    def run(self, population_size=100, mutation_rate=0.01, generations=500, sele
        self.population_size = population_size
        self.mutation_rate = mutation_rate
        self.generations = generations
        self.population = self._create_initial_population()

        for gen in range(generations):
            fitness_scores = [self._calculate_fitness(r) for r in self.populatio
            best_idx = np.argmax(fitness_scores)
            current_best_route = self.population[best_idx][:]
            current_best_distance = 1 / fitness_scores[best_idx]
            self.history.append(current_best_distance)

            if self.best_route is None or current_best_distance < (1 / self._cal
                self.best_route = current_best_route[:]

            line_objects = self._get_route_geometry(current_best_route)
            city_names = [self.citiesGDF.iloc[idx]["city"] for idx in current_be
            if line_objects:
                self.best_geometries_history.append({
                    "generation": gen,
                    "distance_km": current_best_distance,
                    "route_order": " -> ".join(city_names),
                    "geometry": MultiLineString(line_objects),
                })

            new_population = [self.best_route[:]]
            while len(new_population) < self.population_size:
                p1 = self._selection(fitness_scores, method=selection, tournamen
                p2 = self._selection(fitness_scores, method=selection, tournamen
                child = self._crossover(p1, p2)
                child = self._mutate(child)
                new_population.append(child)
            self.population = new_population

    def get_history_gdf(self):
        if not self.best_geometries_history:
            return None
        return gpd.GeoDataFrame(self.best_geometries_history, crs="EPSG:4326")

    def get_results(self):
        if self.best_route is None:
            return None
        city_names = [self.citiesGDF.iloc[idx]['city'] for idx in self.best_rout
        total_distance = 1 / self._calculate_fitness(self.best_route)
        line_objects = self._get_route_geometry(self.best_route)
        combined_line = MultiLineString(line_objects) if line_objects else None
        data = [{
            'distance_km': total_distance,
            'route_order': " -> ".join(city_names),
            'geometry': combined_line
        }]
        return gpd.GeoDataFrame(data, crs="EPSG:4326")
```

4.3. Definicja problemu i inicjalizacja algorytmu

Tworzymy listę `polish_cities` zawierającą 22 miasta rozsiane po różnych regionach Polski (od nadmorskiego Kołobrzegu i Szczecina, przez centralną Warszawę i Łódź, aż po górzyste Lesko i południowy Kraków). Taki rozkład punktów sprawia, że problem staje się wysoce złożony. Liczba wszystkich możliwych unikalnych tras dla 22 miast wynosi $22!$ (ponad 1.1×10^{21} kombinacji). Sprawdzenie każdej z nich tradycyjną metodą przeglądu zupełnego (brute-force) zajęłoby współczesnym komputerom tysiące lat, co doskonale uzasadnia konieczność użycia heurystyki ewolucyjnej.

Następnie tworzymy instancję głównej klasy algorytmu: `tsp = TSPGenetic(polish_cities)`. Przypisanie to uruchamia procesy zdefiniowane w konstruktorze klasy:

1. Skrypt łączy się z zewnętrznym API geolokalizacyjnym, aby pobrać dokładne współrzędne dla każdego miasta.
2. Odpytuje system wyznaczania tras (OSRM) o rzeczywiste, drogowe odległości pomiędzy każdą możliwą parą miast i zapisuje ich kształt (geometrię).
3. Buduje i zapisuje w pamięci kompletną macierz odległości, z której algorytm będzie wielokrotnie korzystał podczas ewolucji bez konieczności ponownego łączenia się z internetem.

In [ ]:
```
polish_cities = ["Szczecin", "Gdansk", "Olsztyn", "Bialystok", "Lublin", "Rzeszo
tsp = TSPGenetic(polish_cities)
```

4.4. Optymalizacja hiperparametrów algorytmu (Grid Search)

Poniższy fragment kodu odpowiada za systematyczne przeszukiwanie przestrzeni hiperparametrów (tzw. Grid Search) w celu znalezienia takiej konfiguracji algorytmu genetycznego, która najskuteczniej minimalizuje dystans na trasie.

Co dokładnie dzieje się w kodzie:

- Definicja siatki parametrów (`param_grid`): Określamy różne wartości dla kluczowych mechanizmów ewolucji: rozmiaru populacji (50, 100, 150), wskaźnika mutacji (1%, 5%, 10%), metody doboru rodziców (turniejowa, ruletkowa, rankingowa) oraz rozmiaru turnieju (3, 5).
- Generowanie kombinacji: Wykorzystując bibliotekę `itertools`, skrypt tworzy iloczyn kartezjański, czyli listę wszystkich możliwych kombinacji tych ustawień.
- Główna pętla testowa: Dla każdej z wygenerowanych kombinacji od nowa tworzona jest instancja algorytmu (`TSPGenetic`), a proces ewolucji uruchamiany jest na dystansie 200 pokoleń.
- Śledzenie najlepszego wyniku: Z każdym przebiegiem kod sprawdza, czy uzyskany całkowity dystans jest mniejszy od dotychczasowego rekordu (`best_distance`). Jeśli tak, zapamiętuje zarówno same parametry, jak i pełną historię geometrii dla tej próby. Dodatkowo wyniki wszystkich iteracji są zbierane do zbiorczej tabeli `results_df`, co pozwala na późniejszą analizę wpływu parametrów na zbieżność.
- Eksport najlepszej ścieżki: Zwycięska (najkrótsza) historia generacji jest na koniec zapisywana do pliku `najlepsza_historia2.geojson`, z którego wygenerowana

zostanie ostateczna animacja.

Środowisko i czas wykonania obliczeń: Z uwagi na bardzo dużą liczbę kombinacji, konieczność symulowania setek tysięcy operacji krzyżowań i mutacji oraz obciążenie wynikające z wielokrotnego odpytywania API geolokalizacyjnego (OSRM), proces ten jest ekstremalnie złożony obliczeniowo. Aby nie przerywać pracy algorytmu, powyższy kod został uruchomiony na zewnętrznym serwerze obliczeniowym z systemem Ubuntu. Całkowity czas egzekucji tego bloku oraz znalezienia optymalnych hiperparametrów wyniósł 6 godzin.

```
param_grid = {
    'population_size': [50, 100, 150],
    'mutation_rate': [0.01, 0.05, 0.1],
    'selection': ['tournament', 'roulette', 'ranking'],
    'tournament_size': [3, 5]
}

keys, values = zip(*param_grid.items())
combinations = [dict(zip(keys, v)) for v in itertools.product(*values)]

best_distance = float('inf')
best_params = None
best_result_gdf = None
best_history_gdf = None
all_results = []

for params in combinations:

    tsp = TSPGenetic(polish_cities)

    tsp.run(
        population_size=params['population_size'],
        mutation_rate=params['mutation_rate'],
        generations=200,
        selection=params['selection'],
        tournament_size=params['tournament_size']
    )

    results = tsp.get_results()
    dist = results.iloc[0]['distance_km']

    all_results.append({
        'population_size': params['population_size'],
        'mutation_rate': params['mutation_rate'],
        'selection': params['selection'],
        'tournament_size': params['tournament_size'],
        'distance': dist
    })

    if dist < best_distance:
        best_distance = dist
        best_params = params
        best_result_gdf = results
        best_history_gdf = tsp.get_history_gdf()
```

```
results_df = pd.DataFrame(all_results)

best_history_gdf.to_file("najlepsza_historia.geojson",
driver="GeoJSON")
```

4.5. Wczytanie mapy Polski

Ten kod wykorzystuje bibliotekę `geopandas` do załadowania granic terytorialnych Polski z lokalnego pliku przestrzennego `data.gpkg` udostępnionego na zajęciach "Analiza danych przestrzennych" w poprzednim semestrze. Wywołanie samej zmiennej `poland` na końcu wyświetla podgląd wczytanej tabeli (w tym kolumnę z geometrią), która posłuży nam w kolejnych krokach jako tło do rysowania wyewoluowanych tras.

In [5]:
```
poland = gpd.read_file("data.gpkg", layer='poland')
poland
```

Out[5]:

	Nazwa	geometry
0	Polska	POLYGON ((495817.822 183473.276, 495778.757 18...

4.6. Analiza wyników optymalizacji

Poniższy kod wczytuje zapisane wcześniej wyniki testów z pliku `params.csv` do ramki danych (DataFrame). Następnie sortuje całą tabelę rosnąco według kolumny `distance` i resetuje indeksy. Dzięki temu zabiegowi na samym szczycie wyświetlonego zestawienia od razu widać tę kombinację parametrów (rozmiar populacji, wskaźnik mutacji, metoda selekcji), która wygenerowała najkrótszą możliwą trasę.

In [11]:
```
parameters = pd.read_csv("params.csv")
parameters.sort_values(by='distance', ascending=True, inplace=True)
parameters.reset_index(drop=True, inplace=True)
parameters
```

Out[11]:

	population_size	mutation_rate	selection	tournament_size	distance
0	100	0.05	ranking	3	2934.1299
1	150	0.10	tournament	5	2937.2508
2	100	0.01	ranking	3	2954.5498
3	150	0.05	ranking	3	3022.1104
4	150	0.05	ranking	5	3065.4950
5	150	0.01	ranking	3	3195.1284
6	150	0.01	tournament	5	3220.1141
7	150	0.01	ranking	5	3232.5388
8	100	0.01	ranking	5	3252.2495
9	150	0.05	tournament	3	3288.7251
10	150	0.01	tournament	3	3341.8736
11	50	0.05	tournament	3	3345.7218
12	100	0.05	ranking	5	3377.2783
13	100	0.10	ranking	3	3392.0646
14	100	0.10	ranking	5	3417.2993
15	50	0.10	tournament	5	3430.5086
16	100	0.10	tournament	5	3442.0196
17	100	0.05	tournament	3	3499.6676
18	50	0.05	roulette	5	3573.8071
19	50	0.05	ranking	3	3613.0569
20	50	0.10	ranking	5	3618.3519
21	50	0.01	ranking	3	3623.7870
22	100	0.10	tournament	3	3678.7178
23	100	0.01	tournament	5	3701.8344
24	150	0.10	ranking	3	3722.0637
25	150	0.10	ranking	5	3738.7740
26	50	0.05	ranking	5	3747.2899
27	100	0.05	tournament	5	3801.5014
28	150	0.10	tournament	3	3881.4097
29	50	0.01	tournament	5	3899.0038
30	50	0.01	roulette	3	3909.6791
31	50	0.10	ranking	3	3920.0264
32	50	0.10	roulette	5	3983.4242

	population_size	mutation_rate	selection	tournament_size	distance
33	50	0.05	tournament	5	3993.6053
34	50	0.10	tournament	3	3995.8048
35	50	0.01	ranking	5	4136.0699
36	150	0.05	tournament	5	4244.6844
37	150	0.01	roulette	5	4314.9991
38	100	0.01	tournament	3	4491.2327
39	100	0.05	roulette	5	4651.1955
40	100	0.05	roulette	3	4651.3837
41	150	0.05	roulette	5	4665.6749
42	150	0.05	roulette	3	4853.2979
43	50	0.01	tournament	3	4853.6009
44	100	0.10	roulette	5	4875.3313
45	100	0.01	roulette	3	4928.1250
46	150	0.01	roulette	3	4949.4961
47	50	0.05	roulette	3	5059.9620
48	100	0.01	roulette	5	5072.3974
49	150	0.10	roulette	5	5094.6929
50	150	0.10	roulette	3	5216.1484
51	50	0.10	roulette	3	5321.1623
52	50	0.01	roulette	5	5342.7100
53	100	0.10	roulette	3	5491.8021

Na podstawie posortowanych wyników z pliku `params.csv` możemy zaobserwować, jak drastyczny wpływ na działanie algorytmu mają jego hiperparametry:

- Najlepszy wynik (ok. 2934 km): Najkrótszą trasę udało się wyznaczyć dla populacji 100 osobników, przy mutacji na poziomie 5% (`0.05`) oraz selekcji rankingowej. Selekcja rankingowa w połączeniu z umiarkowaną mutacją zapewniła optymalny balans – skutecznie faworyzowała elitarne osobniki, zapobiegając jednocześnie przedwczesnemu utknięciu w minimum lokalnym.

- Najgorszy wynik (ok. 5491 km): Najsłabiej (trasa niemal dwukrotnie dłuższa) poradził sobie wariant z tą samą populacją (100), ale wysoką mutacją 10% (`0.1`) i selekcją ruletkową. Takie połączenie wprowadziło zbyt wiele losowości. Zbyt częste mutacje połączone z wysoce probabilistyczną ruletką sprawiły, że ewolucja stała się chaotyczna i algorytm na bieżąco niszczył (gubił) wypracowane dobre fragmenty tras.

4.7. Wygenerowanie ostatecznej animacji na mapie Polski

Ostatnim etapem projektu jest stworzenie wizualizacji dla najlepszej znalezionej trasy. Kod ten generuje animację GIF, nakładając historyczne przebiegi ewolucji bezpośrednio na rzeczywistą mapę terytorium Polski.

Kluczowe kroki realizowane przez ten fragment:

- Projekcja przestrzenna (CRS): Kod pobiera najlepsze wyniki z pliku GeoJSON, a następnie dokonuje transformacji współrzędnych do układu `EPSG:2180` (oficjalny układ współrzędnych PUWG 1992, dedykowany dla Polski). Zapewnia to zachowanie prawidłowych proporcji kształtu kraju oraz wiarygodne wizualnie odwzorowanie odległości na płaskim wykresie.
- Przygotowanie warstw wizualnych: W pętli, dla każdej zapisanej generacji, tworzony jest wielowarstwowy wykres:
  - Na samym dole rysowane są kontury Polski jako tło.
  - Następnie nanoszone są wszystkie miasta, z zachowaniem ich poprawnej lokalizacji.
  - Na to nakładana jest czerwona linia reprezentująca sieć połączeń wypracowaną w danej generacji.
  - Wyznaczony zostaje fioletowy punkt oznaczający start i metę trasy.
- Zarządzanie klatkami i legenda: Zdefiniowana została niestandardowa legenda, a każdy wykres otrzymuje dynamiczny tytuł ze statystykami (numer pokolenia i obecny najkrótszy dystans). Gotowy obraz jest zapisywany jako plik `.png` w folderze `klatki_animacji`.
- Kompozycja animacji: Podobnie jak we wcześniejszych etapach, biblioteka `PIL` pobiera wszystkie wygenerowane klatki i łączy je w płynny plik `ewolucja_trasy.gif`. Z uwagi na to, że wykorzystujemy do podkładu precyzyjne mapy i przekształcenia geometryczne obiektów, to właśnie ta animacja stanowi finalne i najbardziej satysfakcjonujące podsumowanie pracy algorytmu ewolucyjnego.

In [6]:
```
os.makedirs("klatki_animacji", exist_ok=True)

results_all = gpd.read_file("najlepsza_historia2.geojson")
results_all.to_crs("EPSG:2180", inplace=True)
results_all.sort_values(by='generation', ascending=True, inplace=True)
results_all.reset_index(drop=True, inplace=True)

cities_proj = tsp.citiesGDF.to_crs("EPSG:2180")

sciezki_plikow = []

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label='Miasta', markerfacecolor='blu
    Line2D([0], [0], marker='o', color='w', label='Start / Meta', markerfacecolo
    Line2D([0], [0], color='red', lw=2, label='Trasa')
]

for i in range(len(results_all)):
    fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
```

```
    poland.plot(ax=ax, color='lightgray', edgecolor='black')

    cities_proj.plot(ax=ax, color='blue', marker='o', markersize=40, zorder=4)

    current_route = results_all.iloc[[i]]
    current_route.plot(ax=ax, color='red', linewidth=2, zorder=3)

    geom = current_route.iloc[0].geometry
    if geom is not None:
        start_coords = geom.geoms[0].coords[0]
        ax.plot(start_coords[0], start_coords[1], marker='o', color='purple', ma

    dist = current_route.iloc[0]['distance_km']
    gen = current_route.iloc[0]['generation']

    ax.set_title(f"Pokolenie: {gen} | Dystans: {dist:.2f} km", fontsize=16)

    ax.legend(handles=legend_elements, loc='center left', bbox_to_anchor=(1, 0.5

    ax.axis('off')

    nazwa_pliku = f"klatki_animacji/frame_{i:04d}.png"
    plt.savefig(nazwa_pliku, bbox_inches='tight')
    plt.close(fig)

    sciezki_plikow.append(nazwa_pliku)

obrazy = [Image.open(plik) for plik in sciezki_plikow]

if obrazy:
    obrazy[0].save(
        "ewolucja_trasy.gif",
        save_all=True,
        append_images=obrazy[1:],
        duration=500,
        loop=0
    )
```

No description has been provided for this image

5. Podsumowanie

Projekt ten z powodzeniem zademonstrował skuteczność algorytmów ewolucyjnych w rozwiązywaniu złożonych problemów optymalizacyjnych z klasy NP-trudnych, takich jak problem komiwojażera (TSP). Wyznaczenie absolutnie najkrótszej trasy między 22 miastami za pomocą przeglądu zupełnego było niemożliwe z powodu gigantycznej złożoności obliczeniowej (ponad 10^{21} kombinacji). Jednakże zastosowanie mechanizmów inspirowanych naturalną ewolucją – selekcji, krzyżowania i mutacji – pozwoliło na wygenerowanie wysoce zoptymalizowanego przybliżenia w akceptowalnym czasie. Przeprowadzona optymalizacja hiperparametrów uwidoczniła, jak krytyczny dla działania algorytmu jest odpowiedni balans między eksploracją (przeszukiwaniem nowych rozwiązań) a eksploatacją (wykorzystywaniem już znalezionych, dobrych genów). Zaobserwowano, że optymalne wyniki osiągnięto przy użyciu selekcji rankingowej i umiarkowanej mutacji rzędu 5%. Z kolei zbyt wysoki wskaźnik mutacji (10%) połączony z

silnie probabilistyczną selekcją ruletkową prowadził do chaosu i degradowania wypracowanych już, dobrych fragmentów trasy. Trasę najkrótszą doskonale zobrazowano za pomocą wygenerowanej animacji przestrzennej. Potwierdziła ona wizualnie, w jaki sposób algorytm genetyczny z pokolenia na pokolenie "rozplątuje" początkowo losową pajęczynę połączeń, stopniowo zbiegając do uporządkowanego, geometrycznie optymalnego obwodu. Ostatecznie projekt udowodnił, że heurystyki ewolucyjne są potężnym i elastycznym narzędziem inteligencji obliczeniowej. Stanowią one niezastąpioną alternatywę w analizie danych wszędzie tam, gdzie przestrzeń poszukiwań jest zbyt rozległa dla klasycznych metod analitycznych i deterministycznych.